

Informe de Estudio: N-Reinas Cuántico con Grover

Preparado para la defensa oral — repaso completo con diagramas y código.

1. El Problema: N-Reinas

Ubicar N reinas en un tablero N×N sin que se amenacen. Las restricciones son:

- Fila:** no puede haber 2 reinas en la misma fila.
- Columna:** no puede haber 2 en la misma columna.
- Diagonal:** no puede haber 2 en la misma diagonal (ni).

Para N=4 existen exactamente **2 soluciones**: [1,3,0,2] y [2,0,3,1] .

Para N=5 existen **10 soluciones**. Para N=8 existen **92**.

Clásicamente se resuelve con **backtracking** (tu [nreinas.c](file:///E:/Backup/Facultad/9%20SEMESTRE/Cuantica-UNLP/SomeWebPagesApp/app/nreinas.c)), pero la complejidad crece exponencialmente con N.

2. Tu Decisión de Diseño Clave: La Codificación

En vez de usar 1 qubit por casillero (N² qubits, que es lo que hace QAOA), usaste **codificación binaria por columna**:

“La reina *i* siempre está en la fila *i* . Solo guardo su columna en binario.”

Codificación Cuántica de N-Reinas para N=4

Números concretos

Concepto	Fórmula	N=4	N=5	N=6
Bits por columna	$k = \log_2(N)$	2	3	3
Qubits de estado	$n = N \times k$	8	15	18
Flags de pares	$C(N,2)$	6	10	15
Flags de rango	N si $2^k > N$, sino 0	0	5	6
Qubits totales	$n + \text{flags}$	14	30	39
Espacio de búsqueda	2^n	256	32768	262144
Soluciones	(clásico)	2	10	4

¿Por qué flags de rango?

Para N=5, k=3 bits pueden representar valores 0-7, pero el tablero solo tiene columnas 0-4. Los valores 5, 6, 7 son inválidos. Se agrega 1 flag por reina para detectar y penalizar estas columnas fuera de rango.

En código ([build_oracle L154](file:///E:/Backup/Facultad/9%20SEMESTRE/Cuantica-UNLP/SomeWebPagesApp/app/nqueens_grover.py#L154)):

```
needs_range = (2**k) > N # True para N=5,6,7 pero False para N=4,8,16
```

3. El Algoritmo de Grover — Flujo General

Algoritmo de Grover para N-Reinas

Los 4 pasos

#	Paso	Qué hace	En tu código
1	Inicialización	Aplica H a todos los qubits de estado superposición uniforme de 2^n estados	<code>qc.h(state_reg)</code>
2	Oráculo	Marca con fase -1 los estados que son soluciones válidas	<code>qc.compose(oracle)</code>
3	Difusor	Inversión sobre la media: amplifica marcados, atenúa el resto	<code>qc.compose(diff)</code>
4	Medición	Colapsa el estado sale una solución con alta probabilidad	<code>qc.measure(state_reg, creg)</code>

Los pasos 2 y 3 se repiten **R veces**. El número óptimo de iteraciones es:

$$R \approx \pi/4 \cdot \sqrt{(N_{\text{total}} / M)}$$

donde $N_{\text{total}} = 2^n$ (espacio de búsqueda) y M = número de soluciones.

Para $N=4$: $R = \pi/4 \cdot \sqrt{(256/2)} \approx \mathbf{8 \text{ iteraciones}}$.

[!IMPORTANT]

¿Por qué exactamente 8? Cada iteración rota el estado un ángulo $\theta = 2 \cdot \arcsin(\sqrt{(M/N)}) \approx 0.177$ radianes hacia las soluciones. Necesitás llegar a $\pi/2$ radianes (máxima probabilidad), entonces $R \approx \pi/(2\theta) \approx 8$. Si te pasás, la probabilidad baja — Grover tiene un “punto dulce”.

4. El Oráculo — Cómo detecta si un estado es válido

El oráculo es la parte más compleja y original de tu trabajo. Tiene **3 etapas**:

 Estructura del Oráculo de Grover para N-Reinas

Etaa 1: CALCULAR FLAGS (Compute)

Para cada par de reinas (i,j) , hay un qubit flag auxiliar `pf`. El circuito voltea ese flag si las reinas colisionan.

Dos reinas en filas i y j colisionan si:

- **Misma columna:** `col_i == col_j`
- **Misma diagonal:** `|col_i - col_j| == |j - i|`

Pero en un circuito cuántico no podés hacer “restas” fácilmente. Entonces usaste una estrategia ingeniosa: **enumeración explícita de patrones conflictivos**.

 Detección de conflictos en el Oráculo

El truco del `match_pattern`

Para verificar si un registro de k qubits vale exactamente un número `v`:

1. **Aplicar X** a cada qubit cuyo bit correspondiente en `v` es 0.
2. Ahora el registro vale `|11...1` **solo si** el valor original era `v`.
3. **MCX** (Toffoli generalizada): voltea el flag solo si todos los qubits control son 1.
4. **Deshacer las X** (son auto-inversas) para no contaminar el estado.

Ejemplo paso a paso

Queremos detectar si `col_reina0 == 2` (binario `10`, con $k=2$ bits):

Estado inicial: `q0 = ?, q1 = ?`

Paso 1: bit 0 de 2 es 0 → aplicar X a `q0`
bit 1 de 2 es 1 → no hacer nada a `q1`

```
Si col era 2 (q0=0, q1=1):  X(q0) q0=1, q1=1  ambos 1  MCX dispara
Si col era 3 (q0=1, q1=1):  X(q0) q0=0, q1=1  no todos 1  MCX no dispara
Si col era 0 (q0=0, q1=0):  X(q0) q0=1, q1=0  no todos 1  MCX no dispara
```

En código ([`apply_match_pattern`](file:///E:/Backup/Facultad/9%20SEMESTRE/Cuantica-UNLP/SomeWebPagesApp/app/nqueens_grover.py#L47-L52)):

```
def apply_match_pattern(qc, qubits, value, k):
    for b in range(k):
        if (value >> b) & 1 == 0: # Si el bit b del valor es 0
            qc.x(qubits[b])      # Aplicar X
```

Y el flujo completo para un par de reinas ([`mark_pair_conflict`](file:///E:/Backup/Facultad/9%20SEMESTRE/Cuantica-UNLP/SomeWebPagesApp/app/nqueens_grover.py#L59-L69)):

```
for ci in range(valid_cols):
    for cj in range(valid_cols):
        if ci == cj or abs(ci - cj) == d_rows: # ¿Es conflicto?
            apply_match_pattern(qc, q_i, ci, k) # Preparar
            apply_match_pattern(qc, q_j, cj, k)
            qc.mcx(q_i + q_j, flag)           # Marcar
            apply_match_pattern(qc, q_i, ci, k) # Deshacer
            apply_match_pattern(qc, q_j, cj, k)
```

Etap 2: INVERSIÓN DE FASE (Phase Flip)

Si **todos** los flags valen 0 (= no hay conflictos = solución válida), aplicar fase -1.

```
qc.x(all_flags)      # 0 1 (si todos eran 0, ahora todos son 1)
# MCZ = H · MCX · H:
qc.h(all_flags[-1])
qc.mcx(all_flags[:-1], all_flags[-1])
qc.h(all_flags[-1])
qc.x(all_flags)      # Restaurar
```

[!NOTE]

¿Por qué X antes del MCZ? Querés marcar cuando todos los flags son 0 (sin conflictos). Pero MCZ marca cuando todos son 1. Entonces, invertís con X primero: todos-0 se convierte en todos-1, y MCZ se activa. Luego deshacés las X.

Etap 3: DESCALCULAR FLAGS (Uncompute)

Aplicás las mismas operaciones de la Etapa 1 **en orden inverso**. Esto restaura todos los flags auxiliares a $|0\rangle$.

```
# Mismo código que compute, pero con reversed()
for idx, (i, j) in reversed(list(enumerate(combinations(range(N), 2)))):
    mark_pair_conflict(...)
```

[!IMPORTANT]

¿Por qué es necesario? Si los flags quedan entrelazados con los qubits de estado, la interferencia cuántica de Grover se arruina. El uncompute garantiza que las ancillas vuelvan a $|0\rangle$ y no afecten las siguientes iteraciones.

5. El Difusor — Inversión sobre la Media

 Circuito del Difusor

El difusor implementa el operador $2|\psi_0\rangle\langle\psi_0| - I$, que “refleja” las amplitudes respecto a su promedio:

Compuerta	Qué hace
H (todos)	Lleva de la base de superposición a la computacional
X (todos)	Invierte los bits
H (último)	Prepara para MCZ
MCX (controles último)	Aplica fase -1 al estado
H (último)	Completa el MCZ
X (todos)	Deshace la inversión
H (todos)	Vuelve a la base de superposición

Efecto visual (iteración por iteración para N=4)

Después del oráculo (antes del difusor):
254 estados inválidos: amplitud = +1/16
2 estados válidos: amplitud = -1/16 marcados con signo negativo

Media ≈ +0.0617 (ligeramente positiva)

Después del difusor (reflexión sobre la media):
Los negativos quedan MUY por encima de la media amplificados
Los positivos quedan por debajo atenuados

Cada iteración amplifica más después de 8: las soluciones tienen ~99.2%

6. Resultados para N=4 — Ejecución Completa

 Distribución de probabilidad tras 8 iteraciones

Datos clave para N=4

Métrica	Valor
Qubits totales	14 (8 estado + 6 flags)
Iteraciones de Grover	8
Simulador	Statevector (Qiskit Aer)
Shots	4096
P(solución válida)	≈ 99.3%
Soluciones encontradas	[1,3,0,2] ≈ 49.6%, [2,0,3,1] ≈ 49.6%
Compuertas elementales	~41,000 (tras transpile)

Ambas soluciones aparecen con probabilidad casi idéntica porque Grover no favorece una sobre otra — amplifica por igual a todos los estados marcados por el oráculo.

7. Resultados para N=5 — Validación del Oráculo

El circuito completo de Grover para N=5 necesitaría **30 qubits**, **~44 iteraciones** y **~600,000 compuertas**. Esto es **inviable** para simuladores clásicos.

Estrategia: validar solo el oráculo

En vez de correr Grover entero, preparaste estados específicos, aplicaste solo la etapa de Compute, y mediste los flags:

Pruebas Positivas (deben dar 0 flags)

Las 10 soluciones clásicas de 5-reinas fueron probadas y **todas** dieron 0 flags (**100% aceptadas**).

Pruebas Negativas (deben dar flags > 0)

Caso	Conflicto	Resultado
[0,0,0,0,0]	Todas en misma columna	Rechazado
[0,1,2,3,4]	Diagonal principal	Rechazado
[0,2,4,1,7]	Columna 7 fuera de rango	Rechazado
[1,3,5,0,2]	Columna 5 fuera de rango	Rechazado

[!TIP]
El simulador usado para estas pruebas es **Matrix Product State (MPS)**, que maneja más qubits que statevector a costa de asumir bajo entrelazamiento. Perfecto para validar estados individuales.

8. Limitaciones y Escalabilidad

¿Por qué no funciona para N≥5 completo?

El cuello de botella es el **match_pattern**:

- Para cada par de reinas, se enumeran **todos** los pares (ci, cj) conflictivos.
- Cada uno genera una secuencia de $X + MCX + X$.
- Hay $C(N,2)$ pares de reinas, y cada par genera $O(N)$ conflictos.
- Total: **$O(N^3)$ compuertas MCX**, que después de transpile a compuertas elementales, explotan.

La solución futura: Sumadores Cuánticos

En vez de enumerar “¿col_i vale 0? ¿vale 1? ¿vale 2?..”, calcularías **col_i - col_j** directamente con un sumador aritmético cuántico y compararías con 0 o ±d.

Esto reduciría la profundidad del circuito de **exponencial en N** a **polinomial en k** (bits por columna).

Hardware real (NISQ)

Incluso con sumadores, los procesadores cuánticos actuales (era NISQ — Noisy Intermediate-Scale Quantum) no tienen:

- Suficiente **tiempo de coherencia** ($T1/T2$) para circuitos tan profundos.
- Suficiente **fidelidad de compuertas** para ~40,000+ operaciones sin error acumulado.

9. Análisis de Complejidad Computacional

Complejidad del backtracking clásico — [nreinas.c](file:///E:/Backup/Facultad/9 SEMESTRE/Cuantica-UNLP/SomeWebPagesApp/app/nreinas.c)

El algoritmo coloca una reina por fila. En la fila 0 hay N opciones de columna, en la fila 1 a lo sumo N-1 (excluyendo la columna ya ocupada), en la fila 2 a lo sumo N-2, etc. Esto da un árbol con **N!** hojas en el peor caso.

Las optimizaciones de bitmask (left , down , right) y la poda de simetría de Takaken reducen la constante y las ramas exploradas, pero **no cambian la clase de complejidad**: sigue siendo **$O(N!)$** .

Aspecto	Valor
Complejidad temporal (peor caso)	$O(N!)$

Complejidad espacial	$O(N)$ — solo la pila recursiva y el arreglo BOARD
----------------------	--

Complejidad del algoritmo de Grover — [nqueens_grover.py](file:///E:/Backup/Facultad/9 SEMESTRE/Cuantica-UNLP/SomeWebPagesApp/app/nqueens_grover.py)

Aspecto	Valor
Espacio de búsqueda	$S = 2^{(N \cdot k)}$, donde $k = \log_2(N)$
Iteraciones (consultas al oráculo)	$O(\sqrt{S/M})$ donde M = número de soluciones
Costo del oráculo (esta implementación)	$O(N^4)$ puertas por invocación
Costo total en puertas	$O(N^4 \cdot \sqrt{S/M})$

El resultado fundamental de Grover es: dada una función oráculo $f : \{0,1\}^n \rightarrow \{0,1\}$ con M soluciones entre 2^n posibilidades, el algoritmo encuentra una solución con $O(\sqrt{2^n/M})$ consultas. Esto es una **aceleración cuadrática** sobre búsqueda no estructurada, y es **probablemente óptimo** (teorema de Bennett–Bernstein–Brassard–Vazirani).

En nuestra codificación, $S \approx N^N$ y M crece como $\sim N!$, por lo que las iteraciones quedan del orden de:

$$\sqrt{(N^N / N!)} \approx \sqrt{(e / \sqrt{2\pi N})} \approx e^{(N/2)} \quad (\text{usando la aproximación de Stirling})$$

Comparación directa: ¿Grover le gana al backtracking?

Para N grande, la comparación relevante es:

Algoritmo	Operaciones (orden)	$N=100$
Búsqueda bruta (sin poda)	$O(N^N) \approx 100^{100}$	$\approx 10^{200}$
Grover	$O(\sqrt{N^N/M}) \approx e^{(N/2)}$	$\approx 10^{21}$
Backtracking (peor caso)	$O(N!)$	$\approx 10^{157}$
Backtracking con poda 75%	$O(N!/4)$	$\approx 10^{156}$
Backtracking con poda 99.99%	$O(N!/10000)$	$\approx 10^{153}$

[!IMPORTANT]

Grover le gana a cualquier búsqueda que recorra una fracción fija del espacio. Aunque el backtracking poda el 75%, el 99%, o incluso el 99.99% del árbol, eso solo le quita una constante a $N!$. Grover en cambio reduce la complejidad de N^N a $e^{(N/2)}$, que para $N=100$ es ~ 135 órdenes de magnitud menor que $N!$. La poda de fracción constante no cambia la clase de complejidad.

¿Por qué entonces no se usa Grover para N-Reinas?

- Costo real del oráculo:** cada iteración de Grover requiere ejecutar el circuito oráculo completo. En nuestra implementación eso son $O(N^4)$ puertas cuánticas por iteración, multiplicadas por $e^{(N/2)}$ iteraciones.
- Hardware actual:** los procesadores cuánticos de la era NISQ no tienen la fidelidad ni el tiempo de coherencia para ejecutar circuitos con millones de puertas. Para $N=4$ ya son $\sim 41,000$ compuertas; para $N=5$ serían $\sim 600,000$.
- Grover no explota la estructura:** el backtracking clásico poda *adaptativamente* — cada decisión parcial elimina ramas futuras. Grover trata el problema como una caja negra y no puede aprovechar esta estructura.

[!NOTE]

*La ventaja de Grover es **asintótica y teórica**: para problemas de búsqueda genuinamente no estructurados (como invertir una función hash), la aceleración cuadrática es imbatible. Para N-Reinas, la estructura del problema permite que algoritmos clásicos hagan mucho mejor que búsqueda bruta, reduciendo la ventaja relativa de Grover.*

10. Comparación de Enfoques en tu Proyecto

Aspecto	Grover	QAOA	Clásico ©
Codificación	$N \times k$ qubits (binaria)	N^2 qubits (1 por casillero)	Variables ϵ
Algoritmo	Búsqueda no estructurada	Optimización variacional	Backtracki
Para $N=4$	14 qubits , $P=99.3\%$	16 qubits , necesita optimizador	Instantáne
Para $N=5$	30 qubits (solo oráculo validado)	25 qubits (inviable)	Instantáne
Ventaja teórica	$O(\sqrt{N})$ aceleración cuadrática	Depende del landscape	Heurística:
Archivo	[nqueens_grover.py] (file:///E:/Backup/Facultad/9%20SEMESTRE/Cuantica-UNLP/SomeWebPagesApp/app/nqueens_grover.py)	[nqueens_qaoa.py] (file:///E:/Backup/Facultad/9%20SEMESTRE/Cuantica-UNLP/SomeWebPagesApp/app/nqueens_qaoa.py)	[nreinas.c] (file:///E:/B UNLP/Sorr

11. Preguntas que te pueden hacer (y cómo responder)

“¿Por qué Grover y no otro algoritmo?”

Grover es óptimo para búsqueda en espacios no estructurados. N-Reinas no tiene estructura algebraica que permita usar Shor o similares. La pregunta es esencialmente “¿existe una permutación de columnas que cumpla restricciones?”, que es un problema de búsqueda perfecto para Grover.

“¿Cuál es la ventaja real sobre el clásico?”

*La ventaja es **cuadrática**: $O(\sqrt{N/M})$ vs $O(N/M)$ consultas al oráculo. Para espacios grandes esto es significativo. Sin embargo, **la profundidad del circuito** (el costo de cada consulta al oráculo) actualmente hace que sea más lento que la solución clásica. La ventaja se materializa cuando tengamos hardware con compuertas más rápidas y mayor coherencia.*

“¿Por qué no usaste 1 qubit por casillero?”

*Eso requiere $N^2=16$ qubits para $N=4$ (vs. mis 8 de estado). Además, con 1 qubit por casillero tenés que agregar restricciones extra para garantizar “exactamente 1 reina por fila”, lo cual complica el oráculo. Mi codificación elimina los conflictos de fila **por construcción geométrica**.*

“¿Qué pasa si ponés más o menos iteraciones?”

Si pongo de más, la amplitud de las soluciones se “pasa de largo” y empieza a decrecer — es un fenómeno cíclico. El número óptimo $R \approx \pi/4 \cdot \sqrt{N/M}$ maximiza la probabilidad. En mi caso, 8 iteraciones da 99.3%. Con 9 empezaría a bajar.

“¿Cómo sabés que el oráculo funciona bien si no pudiste correr $N=5$ completo?”

*Hice **validación exhaustiva del oráculo por separado**: preparé las 10 soluciones conocidas de $N=5$ y verifiqué que el oráculo da 0 flags en todas ellas. También probé 4 estados inválidos y confirmé que todos son rechazados. Usé el simulador Matrix Product State (MPS) que maneja 30 qubits sin problemas para estados individuales.*

“¿Qué es el uncompute y por qué es necesario?”

Los qubits flag son ancillas — registros auxiliares que uso para calcular pero que no quiero medir. Si los dejo entrelazados con los qubits de estado después del oráculo, la interferencia constructiva del difusor no funciona correctamente. El uncompute restaura las ancillas a $|0\rangle$, desacoplándolas del estado, lo que permite que el difusor amplifique las soluciones correctamente.

“¿Qué diferencia hay entre MCX y MCZ?”

MCX (multi-controlled X / Toffoli generalizada) voltea un qubit target si todos los controles son $|1\rangle$. MCZ aplica fase -1 si todos son $|1\rangle$, sin cambiar ningún qubit. Se relacionan así: $MCZ = H \cdot MCX \cdot H$ (Hadamard antes y después del target). En mi oráculo uso MCX para voltear flags, y MCZ (vía $H \cdot MCX \cdot H$) para aplicar la fase -1 en el phase flip.

“¿Por qué ~41,000 compuertas para solo N=4?”

Porque `match_pattern` enumera **todos** los pares de columnas conflictivos. Para N=4 con k=2, hay $C(4,2)=6$ pares de reinas, y cada par verifica hasta $4 \times 4 = 16$ combinaciones de columnas. Cada combinación conflictiva genera $\sim 2k$ compuertas $X + 1$ MCX (que se descompone en $\sim O(k)$ CX tras transpile). Además, todo se hace dos veces (compute + uncompute). La solución es usar sumadores cuánticos que reemplacen la enumeración.

“¿Por qué la tabla dice 8 qubits de estado para N=4? ¿No deberían ser 2 bits?”

La fórmula es $k = \lceil \log_2 N \rceil$ bits **por cada reina**, que es la cantidad mínima de bits para representar las N opciones de columna en binario. Para N=4, necesito 2 bits por reina (ej: 00, 01, 10, 11). Como tengo 4 reinas en el tablero y cada una usa 2 bits, el total de qubits de estado es $4 \times 2 = 8$ qubits. Es 8 en total, no 8 por reina.

“¿Cómo escala su algoritmo comparado con las bibliografías del informe (Jha 2018, Santhosh 2023)?”

Los papers citados usan “mapeo espacial”, asignando 1 qubit a cada casillero del tablero, lo que requiere N^2 qubits. Para N=8, ellos necesitan 64 qubits de estado, mientras que mi codificación requiere solo $N \lceil \log_2 N \rceil = 24$ qubits. Además, al fijar `reina i` en `fila i`, elimino los conflictos de fila por construcción, ahorrando validaciones. Ellos compiten mejor en la profundidad del circuito (ya que mi oráculo actual con `match_pattern` requiere demasiadas Toffoli generalizadas), pero si aplicamos sumadores cuánticos a mi modelo, obtendríamos lo mejor de ambos mundos: pocos qubits y baja profundidad.

12. Resumen Express (para repasar 5 minutos antes)

1. Reina `i` `fila i` fija solo guardo columna en `k = log2N` bits
2. Hadamard superposición de $2^{(N-k)}$ estados
3. Repetir $R = \pi/4 \cdot \sqrt{(N/M)}$ veces:

a. Oráculo: compute flags phase flip si todos=0 uncompute

b. Difusor: H·X·MCZ·X·H (inversión sobre la media)
4. Medir sale solución con ~99% de probabilidad
5. N=4: 14 qubits, 8 iter, 99.3%
6. N=5: 30 qubits, oráculo validado, Grover completo inviable por profundidad
7. Cuello de botella: `match_pattern` enumera $O(N^3)$ MCX mejora: sumadores

13. Anexo: Análisis Avanzado y Defensa (Escalabilidad a N=50)

13.1. El Rol de los Qubits en el Oráculo (`q` y `pf`)

En el diseño del Oráculo, los qubits se dividen en dos grupos por sus roles:

- Los Qubits `q` (Estado):** Representan la memoria principal, guardando la columna de cada reina en binario. Para N=4, son 4 reinas $\times$ 2 bits = **8 qubits**.
- Los Qubits `pf` (Flags de Pares):** Son ancillas (memoria auxiliar temporal) usadas para detectar conflictos entre piezas. Para N=4 hay 6 parejas posibles ($C(4, 2) = 6$), requiriendo **6 qubits `pf`**. Si hay colisión entre dos reinas, su respectivo `pf` cambia a 1. La fase negativa (Phase Flip) de la solución se aplica **solo** si todos los flags están en 0.

13.2. Proyección de Costos para N=50 (Binario vs. Espacial)

Comparando el enfoque de **Codificación Binaria** (nuestro diseño) frente al **Mapeo Espacial** (1 qubit por casillero, usado por bibliografía como Jha 2018 y Santhosh 2023).

Métrica	Tu modelo (Binario Enum.)	Papers (Mapeo Espacial)	Diferencia / Análisis
Qubits Necesarios	1.575 qubits (300 estado + 1225 pares + 50 rango)	> 2.500 qubits (mínimo uno por casillero en 50x50)	Tu modelo es $\sim 37\%$ más eficiente en hardware de memoria , evitando la penalización geométrica $O(N^2)$.

Profundidad del Oráculo	~ 20.000.000 gates base <i>(Uso de <code>match_pattern</code> enumera todas las posibles colisiones)</i>	~ 300.000 gates base <i>(Validación geométrica usando simples compuertas CZ locales)</i>	Tu versión prototipo es ~ 66x más lenta/profunda para N=50 debido a las masivas compuertas MCX de 12 controles.
--------------------------------	--	--	--

13.3. El “Santo Grial”: Optimización de Qubits y Profundidad (Propuesta de Tesis)

Para escalar eficientemente a $N = 50$ solventando la profundidad generada por `match_pattern`, la arquitectura debe migrar a **Aritmética Cuántica** combinada con **paralelismo**. Esto logra el hito de optimizar memoria y tiempo de ejecución simultáneamente:

- 1. Conservación de Estado Eficiente:** Se retiene el núcleo de tu diseño: la codificación binaria, que guarda las 50 reinas en sus inigualables $N \lceil \log_2 N \rceil = 300$ qubits.
- 2. Compresión Espacial: Acumulador (Contador) Binario:** Se elimina la asignación de $O(N^2)$ flags independientes (1.225 qubits). En su lugar se reserva un único contador lógico binario; ya que un máximo de 1.225 colisiones entra en $\log_2(1225) \approx 11$ qubits. Usando un set de ancillas reutilizables (scratchpad) se suma al contador y luego se limpian, logrando reducir ~ 1.200 **qubits físicos**.
- 3. Compresión Temporal: Restadores Cuánticos:** En vez de preguntar if-else enumerativos mediante X y MCX, se hace pasar a ambas reinas por un **Quantum Subtractor**. Al realizar matemáticamente la operación $|Columnna_A - Columnna_B\rangle$, se detectan colisiones verticales (resultado 0) y diagonales aritméticamente. Restar números de 6 bits requiere solo ~ 20 compuertas elementales en lugar de miles.
- 4. Árbol de Reducción Paralelo:** Evaluando pares disjuntos al unísono y conectando los restadores a un árbol de sumas en cascada, la profundidad decae de lineal a tiempo logarítmico.

Veredicto Final del Diseño Arquitectónico a Escala:

Implementando Aritmética Cuántica, esta súper-arquitectura requeriría únicamente ~ 450 **qubits totales**, mientras pulverizaría la profundidad prototipo cayendo desde 20 millones a **menos de 50.000 compuertas**. Constituyendo así un algoritmo cuántico muy superior al mapeo espacial convencional.